

Kotlin 04.1 版的 Android 進階功能：Android Google 地圖

來源：<https://codelabs.developers.google.com/codelabs/advanced-android-kotlin-training-maps?hl=zh-tw>

步驟數：12

瞭解如何在 Android Kotlin 應用程式中加入 Google 地圖，並設定其樣式。

目錄

1. [1. 事前準備](#)
2. [2. 應用程式總覽](#)
3. [3. 工作：設定專案並取得 API 金鑰](#)
4. [4. 工作：新增地圖類型](#)
5. [5. 工作：新增標記](#)
6. [6. 工作：設定地圖樣式](#)
7. [7. 工作：新增疊加層](#)
8. [8. 工作：啟用位置追蹤功能](#)
9. [9. 解決方案程式碼](#)
10. [10. 摘要](#)
11. [11. 瞭解詳情](#)
12. [12. 下一個程式碼研究室](#)

1. 事前準備

注意：本程式碼研究室內容已過時，不再進行維護。如要瞭解最新的建議做法，請參考「[Android 基本概念：使用 Compose](#)」或「[適合 Android 開發人員的 Jetpack Compose](#)」課程。如需 View 相關內容，可以參考「[Android Kotlin 基本概念](#)」課程。

注意：如要瞭解地圖的更新內容，請參閱「[在您的 Android 應用程式中新增地圖](#)」。

本程式碼研究室是系列課程的一部分，可引導您將地圖加入應用程式。建議您依序完成所有程式碼研究室，因為這些研究室會逐步介紹各項工作。

本系列程式碼研究室包括：

- [4.1 Android Google 地圖](#)
- [4.2 在地圖上新增地理圍欄](#)

使用 Google 地圖建構應用程式時，您可以為應用程式新增衛星圖像、地圖的強大 UI 控制項、位置追蹤和地點標記等功能。您可以顯示自有資料集中的資訊，例如知名釣魚或攀岩地點，為標準 Google 地圖增添價值。您也可以建立遊戲，讓玩家探索實體世界，例如尋寶遊戲，甚至是擴增實境遊戲。

在本課程中，您將建立名為 Wander 的 Google 地圖應用程式，顯示自訂地圖並顯示使用者位置。

必要條件

瞭解下列事項：

- 如何使用 Android Studio 建立及執行基本的 Android 應用程式。
- 如何建立及管理字串等資源。
- 如何使用 Android Studio 重構程式碼及重新命名變數。
- 如何以使用者身分使用 Google 地圖。
- 如何設定執行階段權限。

課程內容

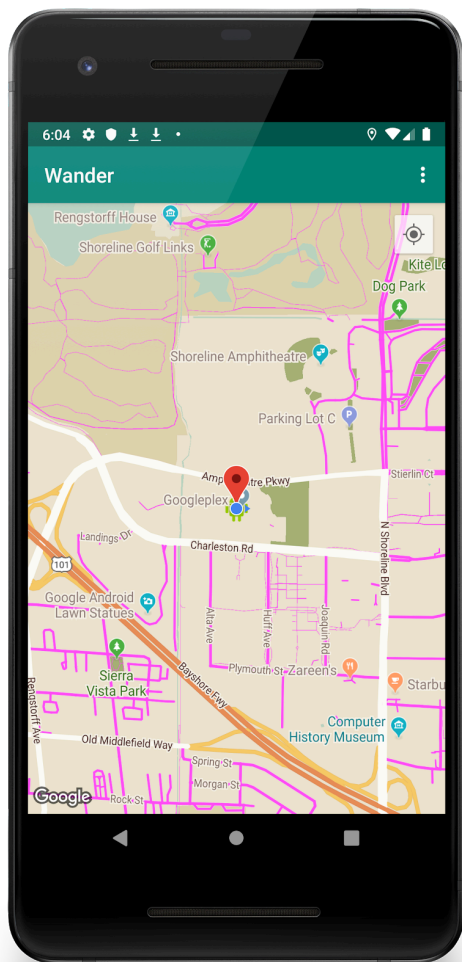
- 如何從 Google API 控制台取得 API 金鑰，並向應用程式註冊金鑰
- 如何在應用程式中整合 Google 地圖
- 如何顯示不同地圖類型
- 如何設定 Google 地圖的樣式
- 如何在 Google 地圖中加入標記
- 如何讓使用者在搜尋點上放置標記
- 如何啟用位置追蹤功能
- 如何建立「The `wander`」應用程式，其中內嵌 Google 地圖
- 如何為應用程式建立自訂功能，例如標記和樣式

- 如何在應用程式中啟用位置追蹤功能
-

步驟 2 · 約 0 分鐘

2. 應用程式總覽

在本程式碼研究室中，您將建立 `Wander` 應用程式，顯示包含自訂樣式的 Google 地圖。Wander 應用程式可讓您在地點上放置標記、新增疊加層，以及即時查看自己的位置。



3. 工作：設定專案並取得 API 金鑰

Maps SDK for Android 需要 API 金鑰。如要取得 API 金鑰，請在「[API 和服務](#)」頁面中註冊專案。API 金鑰會繫結至數位憑證，將應用程式連結至作者。如要進一步瞭解如何使用數位憑證及簽署應用程式，請參閱「[簽署應用程式](#)」。

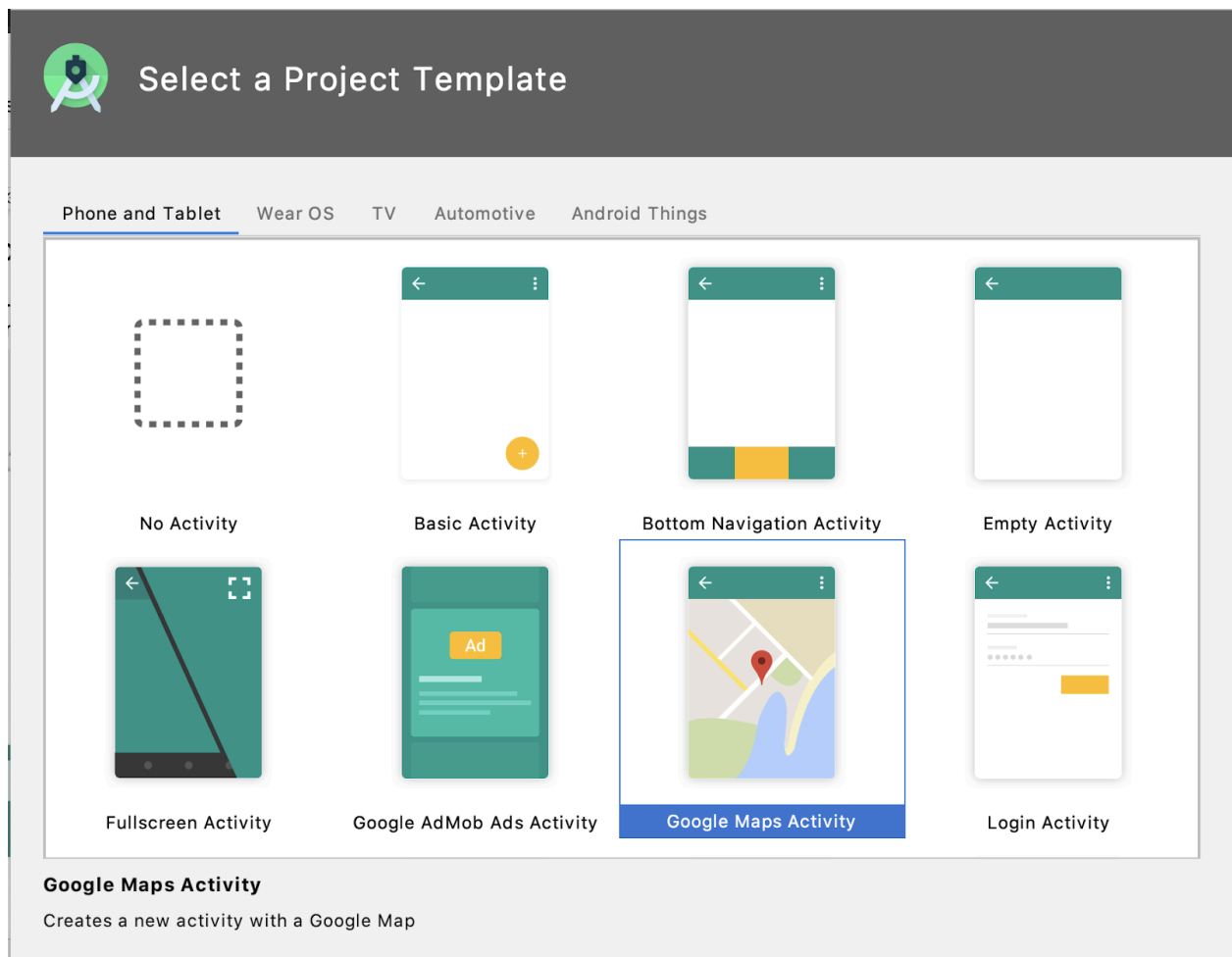
在本程式碼研究室中，您會使用偵錯憑證的 API 金鑰。如「[簽署偵錯版本](#)」一文所述，偵錯憑證的設計本來就不安全，如果發布的 Android 應用程式使用 [Maps SDK for Android](#)，則需要第二組 API 金鑰，也就是發布憑證的金鑰。如要進一步瞭解如何取得發布憑證，請參閱「[取得 API 金鑰](#)」一文。

Android Studio 內含 Google 地圖活動範本，可產生實用的範本程式碼。範本程式碼包含 `google_maps_api.xml` 檔案，內含可簡化 API 金鑰取得程序的連結。

注意：如要建構活動，但不想使用範本，請按照「[取得 API 金鑰](#)」一文中的步驟操作，取得 API 金鑰，而不使用範本中的連結。

步驟 1：使用地圖範本建立 Wander 專案

1. 建立新的 Android Studio 專案。
2. 選取「Google 地圖活動」範本。



3. 將專案命名為 `Wander`。
4. 將最低 API 級別設為 **API 19**。確認語言為 **Kotlin**。
5. 按一下「完成」。

6. 應用程式建構完成後，請查看專案，以及 Android Studio 為您建立的下列地圖相關檔案：

google_maps_api.xml：您可以使用這個設定檔來保存 API 金鑰。範本會產生兩個 **google_maps_api.xml** 檔案，分別用於偵錯和發布。偵錯憑證的 API 金鑰檔案位於 **src/debug/res/values**。發布憑證的 API 金鑰檔案位於 **src/release/res/values**。在本程式碼研究室中，您只會使用偵錯憑證。

activity_maps.xml：這個版面配置檔案包含填滿整個畫面的單一片段。**SupportMapFragment** 類別是 **Fragment** 類別的子類別。**SupportMapFragment** 是在應用程式中放置地圖最簡單的方式，可做為地圖檢視區塊的包裝函式，自動處理必要的生命週期需求。

您可以在任何 **ViewGroup** 中使用 **<fragment>** 標記，並搭配額外的 **name** 屬性，在版面配置檔案中加入 **SupportMapFragment**。

```
android:name="com.google.android.gms.maps.SupportMapFragment"
```

MapsActivity.java：**MapsActivity.kt** 檔案會在 **onCreate()** 方法中例項化 **SupportMapFragment**，並使用類別的 **getMapAsync()** 自動初始化地圖系統和檢視區塊。包含 **SupportMapFragment** 的活動必須實作 **OnMapReadyCallback** 介面和該介面的 **onMapReady()** 方法。地圖載入時，系統會呼叫 **onMapReady()** 方法。

注意：如果現在執行應用程式，地圖會無法載入。查看記錄時，您會看到 API 金鑰設定不正確的訊息。在下一個步驟中，您將取得 API 金鑰，讓應用程式顯示地圖。

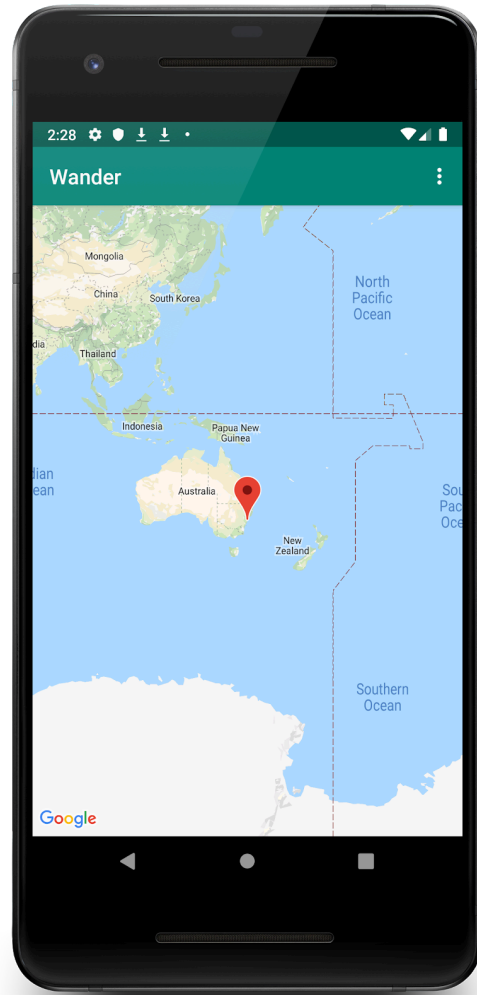
注意：如要在模擬器上測試 Wander 應用程式，必須使用包含 Google Play 的系統映像檔。在虛擬裝置清單的「目標」欄中，選取顯示「Google Play」的圖片。

Type	Name	Play Store	Target
	Pixel 2 API 28		Android 9.0 (Google Play)
	Pixel 3a API 29		Android 10.0 (Google Play)

步驟 2：取得 API 金鑰

1. 開啟 **google_maps_api.xml** 檔案的偵錯版本。
2. 在檔案中找出含有長網址的註解。網址參數包含應用程式的特定資訊。
3. 複製網址並貼到瀏覽器中。
4. 按照提示在「API 和服務」頁面建立專案。由於提供的網址中含有參數，因此頁面會自動啟用 Maps SDK for Android。
5. 按一下「建立 API 金鑰」。
6. 在下一個頁面中，前往「API 金鑰」部分，然後點選您剛才建立的金鑰。
7. 按一下「限制金鑰」，然後選取「Maps SDK for Android」，將金鑰的使用限制在 Android 應用程式。
8. 複製產生的 API 金鑰。開頭為「AIza」。
9. 在 **google_maps_api.xml** 檔案中，將金鑰貼到 **google_maps_key** 字串中標示 **YOUR_KEY_HERE** 的位置。

10. 執行應用程式。您應該會在活動中看到內嵌地圖，並在澳洲雪梨設定標記。(雪梨標記是範本的一部分，您稍後可以變更)。

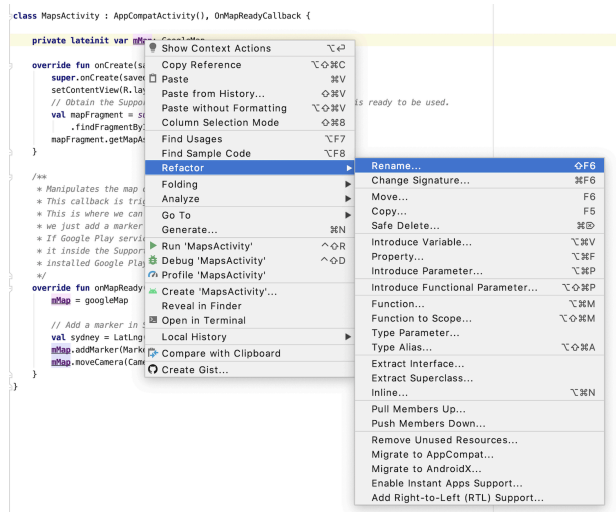


注意：API 金鑰最多可能需要 5 分鐘才會生效。您可能也需要重新啟動 Android Studio。

步驟 3：重新命名 mMap

`MapsActivity` 具有名為 `mMap` 的私有 `lateinit var`，類型為 `GoogleMap`。如要遵循 Kotlin 命名慣例，請將 `mMap` 的名稱變更為 `map`。

1. 在 `MapsActivity` 中，按一下滑鼠右鍵並點選「Refactor」>「Rename...」`mMap`



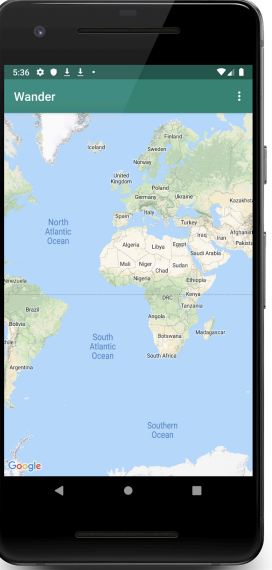
2. 將變數名稱變更為 `map`。

請注意，`onMapReady()` 函式中對 `mMap` 的所有參照也會變更為 `map`。

步驟 4 · 約 0 分鐘

4. 工作：新增地圖類型

Google 地圖提供多種地圖類型：一般、混合、衛星、地形和「無」(完全不顯示地圖)。

			
法線貼圖	衛星地圖	混合地圖	地形地圖

每種地圖提供的資訊類型不同。舉例來說，在車上使用地圖導航時，查看街道名稱很有幫助，因此你可以使用「一般」選項。在健行時，地形圖可協助你判斷還需要爬升多少高度才能抵達山頂。

在這項工作中，您將：

- 1. 新增應用程式列和選項選單，讓使用者變更地圖類型。
- 2. 將地圖的起始位置移至住家位置。
- 3. 新增標記支援，標記可用來指出地圖上的單一位置，並可包含標籤。

新增地圖類型選單

在這個步驟中，您會新增應用程式列和選項選單，讓使用者變更地圖類型。

- 1. 如要建立新的選單 XML 檔案，請在「res」目錄上按一下滑鼠右鍵，然後依序選取「New」 > 「Android Resource File」。
- 2. 在對話方塊中，將檔案命名為 `map_options`。
- 3. 選擇資源類型的「選單」。
- 4. 按一下「確定」。
- 5. 在「Code」分頁中，將新檔案中的程式碼替換成下列程式碼，建立地圖選單選項。由於「無」地圖類型會導致完全沒有地圖，因此已省略。這個步驟會導致錯誤，但您會在下一個步驟中解決這個問題。

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/normal_map"
        android:title="@string/normal_map"
        app:showAsAction="never" />
    <item
        android:id="@+id/hybrid_map"
        android:title="@string/hybrid_map"
        app:showAsAction="never" />
    <item
        android:id="@+id/satellite_map"
        android:title="@string/satellite_map"
        app:showAsAction="never" />
    <item
        android:id="@+id/terrain_map"
        android:title="@string/terrain_map"
        app:showAsAction="never" />
</menu>
```

6. 在 `strings.xml` 中，為 `title` 屬性新增資源，解決錯誤。

```
<resources>
    ...
    <string name="normal_map">Normal Map</string>
    <string name="hybrid_map">Hybrid Map</string>
    <string name="satellite_map">Satellite Map</string>
    <string name="terrain_map">Terrain Map</string>
    <string name="lat_long_snippet">Lat: %1$.5f, Long: %2$.5f</string>
    <string name="dropped_pin">Dropped Pin</string>
    <string name="poi">poi</string>
</resources>
```

7. 在 `MapsActivity` 中，覆寫 `onCreateOptionsMenu()` 方法，並從 `map_options` 資源檔案加載選單。

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    val inflater = menuInflater
    inflater.inflate(R.menu.map_options, menu)
    return true
}
```

8. 在 `MapsActivity.kt` 中，覆寫 `onOptionsItemSelected()` 方法。使用地圖類型常數變更地圖類型，以反映使用者的選取項目。

```

override fun onOptionsItemSelected(item: MenuItem) = when (item.itemId) {
    // Change the map type based on the user's selection.
    R.id.normal_map -> {
        map.mapType = GoogleMap.MAP_TYPE_NORMAL
        true
    }
    R.id.hybrid_map -> {
        map.mapType = GoogleMap.MAP_TYPE_HYBRID
        true
    }
    R.id.satellite_map -> {
        map.mapType = GoogleMap.MAP_TYPE_SATELLITE
        true
    }
    R.id.terrain_map -> {
        map.mapType = GoogleMap.MAP_TYPE_TERRAIN
        true
    }
    else -> super.onOptionsItemSelected(item)
}

```

9. 執行應用程式。

10. 按一下



即可變更地圖類型。請注意地圖在不同模式下的外觀變化。

Map



5. 工作：新增標記

根據預設，`onMapReady()` 回呼包含的程式碼會在 Google 地圖的建立地點 (澳洲雪梨) 放置標記。預設回呼也會將地圖動畫平移至雪梨。

在這項工作中，您要讓地圖的攝影機移至住家位置、縮放至指定層級，並在該處放置標記。

步驟 1：縮放畫面至住家位置並新增標記

1. 在 `MapsActivity.kt` 檔案中，找到 `onMapReady()` 方法。移除程式碼中將標記放在雪梨並移動攝影機的部分。現在的方法應如下所示。

```
override fun onMapReady(googleMap: GoogleMap) {  
    map = googleMap  
  
}
```

2. 按照[這些指示](#)找出住家的經緯度。
3. 建立緯度和經度的值，並輸入浮點值。

```
val latitude = 37.422160  
val longitude = -122.084270
```

4. 建立名為 `homeLatLng` 的新 `LatLng` 物件。在 `homeLatLng` 物件中，傳入您剛才建立的值。

```
val homeLatLng = LatLng(latitude, longitude)
```

3. 建立 `val`，設定地圖的縮放程度。使用縮放比例 `15f`。

```
val zoomLevel = 15f
```

縮放等級會決定地圖的放大程度。以下清單列出各縮放等級大致可顯示的精細程度：

- 1：全球
- 5：陸地/大陸
- 10：城市
- 15：街道
- 20：建築物

4. 在 `map` 物件上呼叫 `moveCamera()` 函式，並使用 `CameraUpdateFactory.newLatLngZoom()` 傳入 `CameraUpdate` 物件，將攝影機移至 `homeLatLng`。傳入 `homeLatLng` 物件和 `zoomLevel`。

```
map.moveCamera(CameraUpdateFactory.newLatLngZoom(homeLatLng, zoomLevel))
```

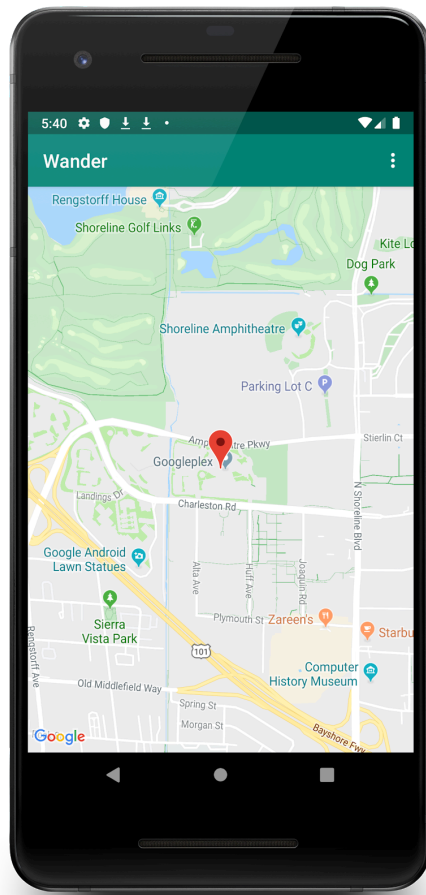
6. 在地圖上的 `homeLatLng` 新增標記。

```
map.addMarker(MarkerOptions().position(homeLatLng))
```

最終的方法應如下所示：

```
override fun onMapReady(googleMap: GoogleMap) {  
    map = googleMap  
  
    //These coordinates represent the latitude and longitude of the Googleplex.  
    val latitude = 37.422160  
    val longitude = -122.084270  
    val zoomLevel = 15f  
  
    val homeLatLng = LatLng(latitude, longitude)  
    map.moveCamera(CameraUpdateFactory.newLatLngZoom(homeLatLng, zoomLevel))  
    map.addMarker(MarkerOptions().position(homeLatLng))  
}
```

7. 執行應用程式。地圖應會平移至住家位置、縮放至所需等級，並在住家位置放置標記。



步驟 2：允許使用者長按新增標記

在這個步驟中，您會在使用者觸控並按住地圖上的位置時新增標記。

1. 在 `MapsActivity` 中建立名為 `setMapLongClick()` 的方法存根，並將 `GoogleMap` 做為引數。
2. 將 `setOnMapLongClickListener` 監聽器附加至地圖物件。

```
private fun setMapLongClick(map:GoogleMap) {  
    map.setOnMapLongClickListener { }  
}
```

3. 在 `setOnMapLongClickListener()` 中呼叫 `addMarker()` 方法。傳入新的 `MarkerOptions` 物件，並將位置設為傳入的 `LatLng`。

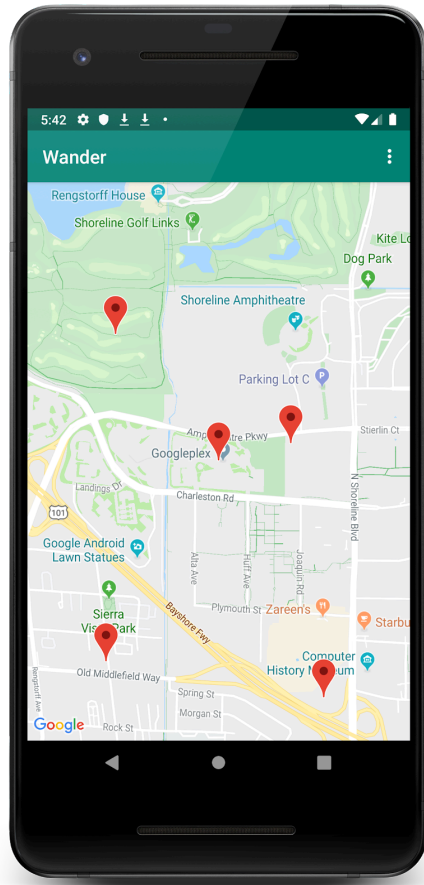
```
private fun setMapLongClick(map: GoogleMap) {  
    map.setOnMapLongClickListener { latLng ->  
        map.addMarker(  
            MarkerOptions()  
                .position(latLng)  
        )  
    }  
}
```

4. 在 `onMapReady()` 方法結尾，使用 `map` 呼叫 `setMapLongClick()`。

```
override fun onMapReady(googleMap: GoogleMap) {  
    ...  
  
    setMapLongClick(map)  
}
```

5. 執行應用程式。
6. 按住地圖，在所需位置放置標記。
7. 輕觸標記，將其置中顯示在畫面上。

注意：輕觸標記時，系統會顯示導航按鈕，方便使用者透過 Google 地圖應用程式導航至標記位置。



步驟 3：為標記新增資訊視窗

在這個步驟中，您要新增 `InfoWindow`，在輕觸標記時顯示標記的座標。

1. 在 `setMapLongClick()``setOnMapLongClickListener()` 中，為 `snippet` 建立 `val`。摘要是顯示在標題後方的額外文字，摘要會顯示標記的經緯度。

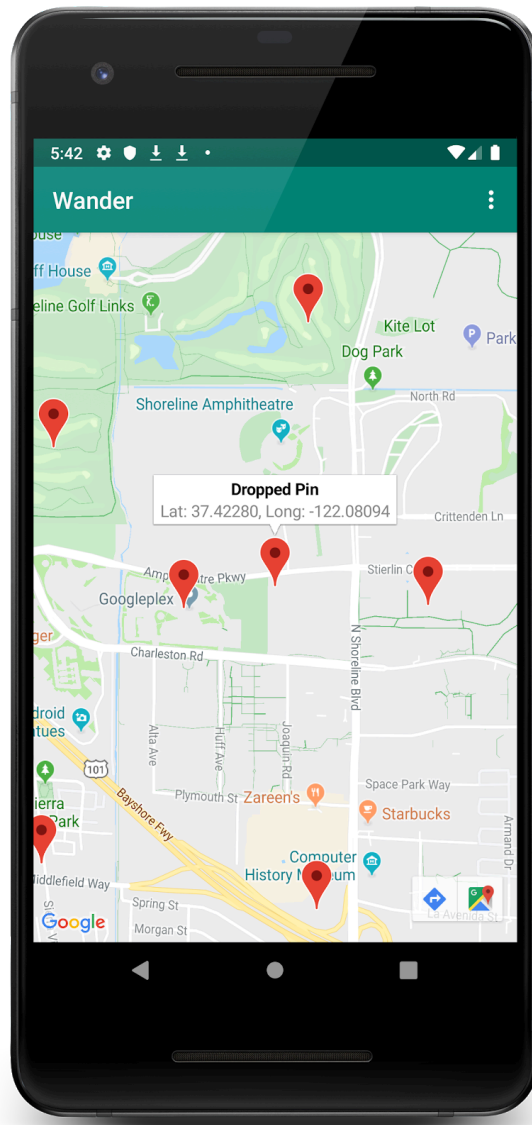
```
private fun setMapLongClick(map: GoogleMap) {
    map.setOnMapLongClickListener { latLng ->
        // A snippet is additional text that's displayed after the title.
        val snippet = String.format(
            Locale.getDefault(),
            "Lat: %1$.5f, Long: %2$.5f",
            latLng.latitude,
            latLng.longitude
        )
        map.addMarker(
            MarkerOptions()
                .position(latLng)
        )
    }
}
```

2. 在 `addMarker()` 中，使用 `R.string.dropped_pin` 字串資源，將標記的 `title` 設為「Dropped Pin」。
3. 將標記的 `snippet` 設為 `snippet`。

已完成的函式如下所示：

```
private fun setMapLongClick(map: GoogleMap) {  
    map.setOnMapLongClickListener { latLng ->  
        // A Snippet is Additional text that's displayed below the title.  
        val snippet = String.format(  
            Locale.getDefault(),  
            "Lat: %1$.5f, Long: %2$.5f",  
            latLng.latitude,  
            latLng.longitude  
        )  
        map.addMarker(  
            MarkerOptions()  
                .position(latLng)  
                .title(getString(R.string.dropped_pin))  
                .snippet(snippet)  
        )  
    }  
}
```

4. 執行應用程式。
5. 按住地圖即可放置地點標記。
6. 輕觸標記即可顯示資訊視窗。



步驟 4：新增 POI 監聽器

根據預設，地圖上會顯示搜尋點和對應的圖示。搜尋點包括公園、學校、政府大樓等。如果地圖類型設為 `normal`，地圖上也會顯示商家搜尋點。商家搜尋點代表商家，例如商店、餐廳和飯店。

在這個步驟中，您會在 Google 地圖中新增 `GoogleMap.OnPoiClickListener`。使用者點選搜尋點時，這個點擊監聽器會立即在地圖上放置標記。點擊事件監聽器也會顯示包含興趣點名稱的資訊視窗。

1. 在 `MapsActivity` 中建立名為 `setPoiClick()` 的方法存根，並將 `GoogleMap` 做為引數。
2. 在 `setPoiClick()` 方法中，對傳入的 `GoogleMap` 設定 `OnPoiClickListener`。

```
private fun setPoiClick(map: GoogleMap) {  
    map.setOnPoiClickListener { poi ->  
  
    }  
}
```

3. 在 `setOnPoiClickListener()` 中，為標記建立 `val poiMarker`。
4. 使用 `map.addMarker()` 將其設為標記，並透過 `MarkerOptions` 將 `title` 設為搜尋點名稱。

```
private fun setPoiClick(map: GoogleMap) {
    map.setOnPoiClickListener { poi ->
        val poiMarker = map.addMarker(
            MarkerOptions()
                .position(poi.latLng)
                .title(poi.name)
        )
    }
}
```

5. 在 `setOnPoiClickListener()` 函式中，對 `poiMarker` 呼叫 `showInfoWindow()`，即可立即顯示資訊視窗。

```
poiMarker.showInfoWindow()
```

`setPoiClick()` 函式的最終程式碼應如下所示。

```
private fun setPoiClick(map: GoogleMap) {
    map.setOnPoiClickListener { poi ->
        val poiMarker = map.addMarker(
            MarkerOptions()
                .position(poi.latLng)
                .title(poi.name)
        )
        poiMarker.showInfoWindow()
    }
}
```

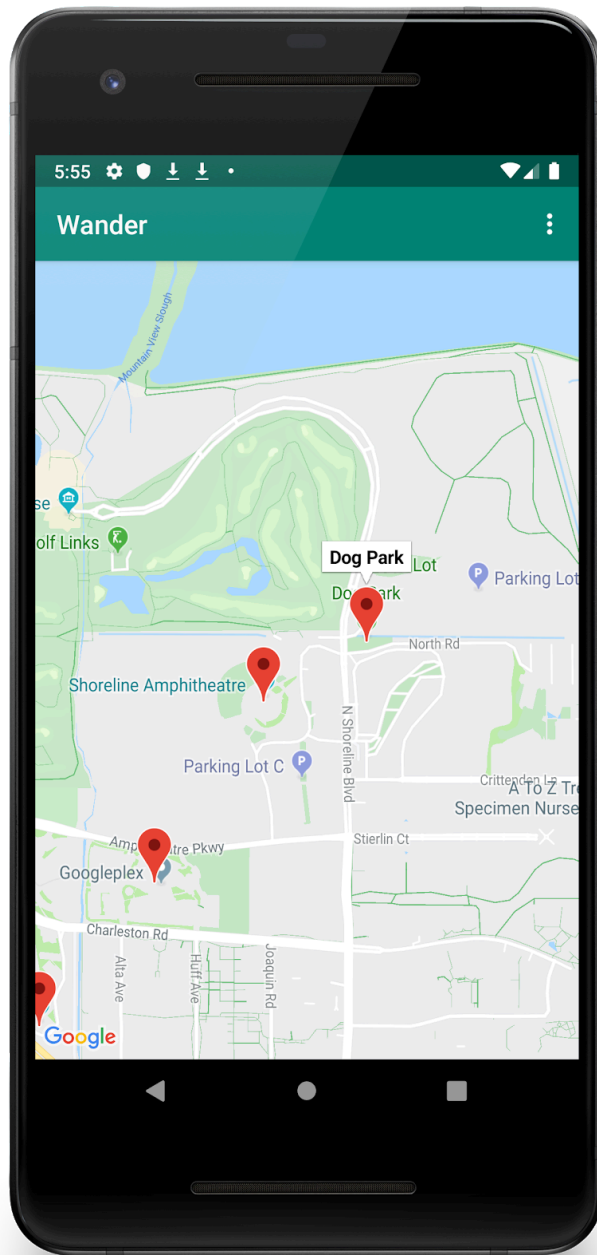
6. 在 `onMapReady()` 結尾呼叫 `setPoiClick()`，並傳入 `map`。

```
override fun onMapReady(googleMap: GoogleMap) {
    ...

    setPoiClick(map)
}
```

7. 執行應用程式並尋找 POI，例如公園或咖啡店。

8. 輕觸興趣點，即可在該處放置標記，並在資訊視窗中顯示興趣點名稱。



6. 工作：設定地圖樣式

你可以透過多種方式自訂 Google 地圖，打造獨特的外觀和風格。

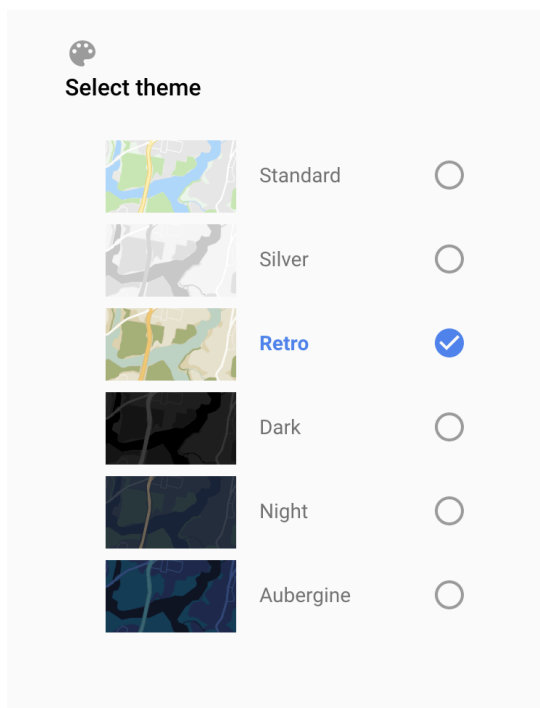
您可以透過可用的 **XML 屬性** 自訂 `MapFragment` 物件，就像自訂任何其他片段一樣。不過，在這個步驟中，您將使用 `GoogleMap` 物件的方法，自訂 `MapFragment` 的內容外觀和風格。

如要為地圖建立自訂樣式，請產生 JSON 檔案，指定地圖中顯示的特徵。您不必手動建立這個 JSON 檔案。Google 提供 [地圖平台樣式精靈](#)，您只要以視覺化方式設定地圖樣式，精靈就會為您產生 JSON。在這項工作中，您將使用復古主題設定地圖樣式，也就是使用復古色彩的地圖，並新增彩色道路。

注意：樣式僅適用於使用 `normal` 地圖類型的地圖。

步驟 1：建立地圖樣式

1. 在瀏覽器中前往 <https://mapstyle.withgoogle.com/>。
2. 選取「建立樣式」。
3. 選取「復古」。



4. 按一下 [更多選項]。

MORE OPTIONS

5. 依序選取「道路」 > 「填滿」。
6. 將道路顏色變更為所選顏色 (例如粉紅色)。

←

Advanced

↶ ⋮

Feature type	Element type	Stylers
Neighborhood > • Land parcel >	All >	Visibility ☒ inherit ☐ hidden ☐ simplified ☐ shown
Landscape > Human-made > • Natural > Landcover > Terrain >	• Geometry > • Fill > Stroke >	☒ Color #ff11ff
• Points of interest > Attraction > Business > Government > Medical > • Park > Place of worship > School > Sports complex >	Labels > Text > Text fill > Text outline > Icon >	☐ Weight
• Road > • Highway > • Controlled access > • Arterial > • Local >		The effect of the following stylers will change whenever Google updates the base map style. i Use with caution.
Transit > • Line > • Station > Airport > Bus > Rail >		☐ Saturation
• Water >		☐ Lightness

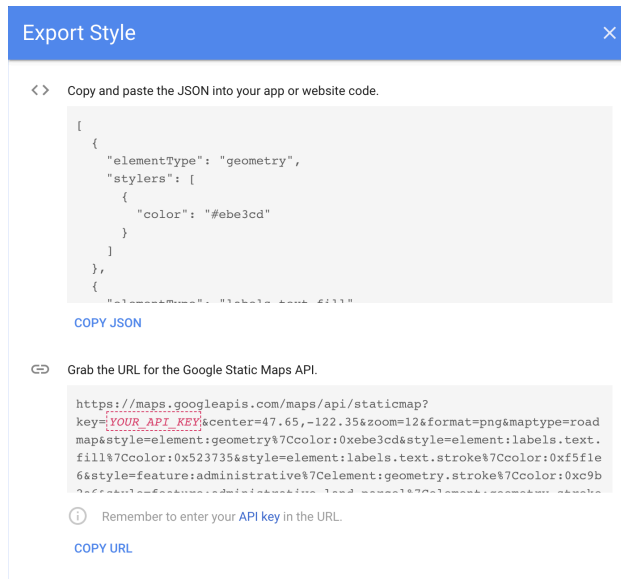
FEWER OPTIONS

FINISH

7. 按一下「完成」。

FINISH

8. 複製結果對話方塊中的 JSON 程式碼，並視需要將其儲存在純文字記事，以供下一個步驟使用。



步驟 2：在地圖上新增樣式

1. 在 Android Studio 的 `res` 目錄中，建立名為 `raw` 的資源目錄。您可以使用 `raw` 目錄資源，例如 JSON 程式碼。
2. 在 `res/raw` 中建立名為 `map_style.json` 的檔案。
3. 將暫存的 JSON 程式碼貼到新的資源檔案中。
4. 在 `MapsActivity` 中，於 `onCreate()` 方法上方建立 `TAG` 類別變數。這會用於記錄。

```
private val TAG = MapsActivity::class.java.simpleName
```

5. 同樣在 `MapsActivity` 中，建立採用 `GoogleMap` 的 `setMapStyle()` 函式。
6. 在 `setMapStyle()` 中新增 `try{}` 區塊。
7. 在 `try{}` 區塊中，建立 `val success` 以順利設定樣式。(您會新增下列 `catch` 區塊)。
8. 在 `try{}` 區塊中，將 JSON 樣式設為地圖，並在 `GoogleMap` 物件上呼叫 `setMapStyle()`。傳遞 `MapStyleOptions` 物件，載入 JSON 檔案。
9. 將結果指派給 `success`。 `setMapStyle()` 方法會傳回布林值，指出剖析樣式檔案及設定樣式的成功狀態。

```
private fun setMapStyle(map: GoogleMap) {
    try {
        // Customize the styling of the base map using a JSON object defined
        // in a raw resource file.
        val success = map.setMapStyle(
            MapStyleOptions.loadRawResourceStyle(
                this,
                R.raw.map_style
            )
        )
    }
}
```

10. 新增 `success` 為 `false` 的 if 陳述式。如果樣式設定失敗，請列印剖析失敗的記錄。

```
private fun setMapStyle(map: GoogleMap) {
    try {
        ...
        if (!success) {
            Log.e(TAG, "Style parsing failed.")
        }
    }
}
```

11. 新增 `catch{}` 區塊，處理缺少樣式檔案的情況。在 `catch` 區塊中，如果無法載入檔案，請擲回 `Resources.NotFoundException`。

```
private fun setMapStyle(map: GoogleMap) {
    try {
        ...
    } catch (e: Resources.NotFoundException) {
        Log.e(TAG, "Can't find style. Error: ", e)
    }
}
```

完成的方法應如下列程式碼片段所示：

```
private fun setMapStyle(map: GoogleMap) {
    try {
        // Customize the styling of the base map using a JSON object defined
        // in a raw resource file.
        val success = map.setMapStyle(
            MapStyleOptions.loadRawResourceStyle(
                this,
                R.raw.map_style
            )
        )

        if (!success) {
            Log.e(TAG, "Style parsing failed.")
        }
    } catch (e: Resources.NotFoundException) {
        Log.e(TAG, "Can't find style. Error: ", e)
    }
}
```

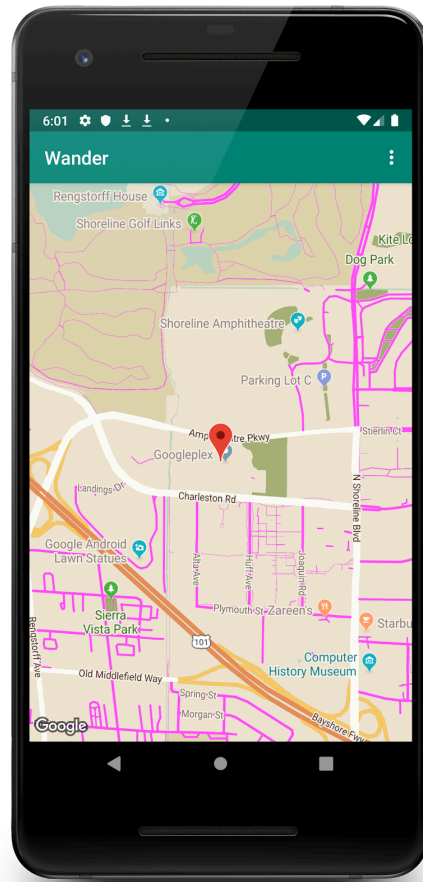
12. 最後，在 `onMapReady()` 方法中呼叫 `setMapStyle()` 方法，並傳入 `GoogleMap` 物件。

```
override fun onMapReady(googleMap: GoogleMap) {
    ...
    setMapStyle(map)
}
```

13. 執行應用程式。

14. 將地圖設為 `normal` 模式，即可看到新樣式，包括復古主題和所選顏色的道路。

注意：如果縮小地圖，地圖就不會再顯示道路，因此您不會看到道路。



步驟 3：設定標記樣式

您可以進一步自訂地圖標記的樣式，打造個人專屬地圖。在這個步驟中，您會將預設的紅色標記變更為更酷的樣式。

1. 在 `onMapLongClick()` 方法中，將下列程式碼行新增至建構函式的 `MarkerOptions()`，使用預設標記，但將顏色變更為藍色。

```
.icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_BLUE))
```

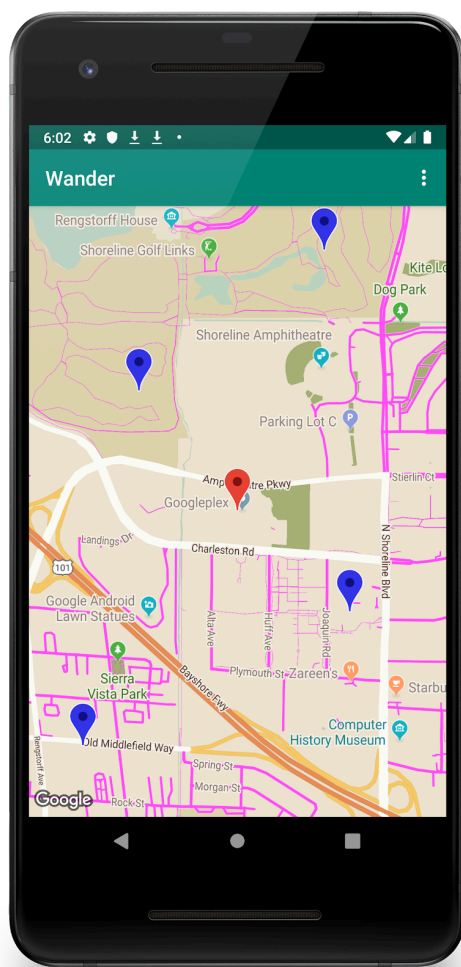
現在，`onMapLongClickListener()` 看起來會像這樣：

```

map.setOnMapLongClickListener { latLng ->
    // A snippet is additional text that's displayed after the title.
    val snippet = String.format(
        Locale.getDefault(),
        "Lat: %1$.5f, Long: %2$.5f",
        latLng.latitude,
        latLng.longitude
    )
    map.addMarker(
        MarkerOptions()
            .position(latLng)
            .title(getString(R.string.dropped_pin))
            .snippet(snippet)
            .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_BLUE))
    )
}

```

2. 執行應用程式。長按後顯示的標記現在會以藍色陰影呈現。請注意，由於您未在 `onPoiClick()` 方法中加入樣式，因此搜尋點標記仍為紅色。



7. 工作：新增疊加層

自訂 Google 地圖的方法之一，就是在地圖上繪製內容。如果您想強調特定類型的地點 (例如熱門釣魚地點)，這個技巧就十分實用。

- **形狀**：您可以在地圖上新增折線、多邊形和圓形。
- **GroundOverlay 物件**：區域疊加層是指固定於地圖上的圖片。不同於標記，區域疊加層的顯示方向是依據實際地表而非裝置螢幕。旋轉、傾斜或縮放地圖時，圖片的方向也會隨之改變。當您想修正地圖上某個區域的單一圖片時，區域疊加層就非常實用。

步驟：新增區域疊加層

在這項工作中，您要在住家位置新增 Android 形狀的地面疊加層。

1. 下載 [這個 Android 映像檔](#)，並儲存在 `res/drawable` 資料夾中。(請確認檔案名稱為 `android.png`)。



2. 在 `onMapReady()` 中，呼叫將攝影機移至住家位置後，請建立 `GroundOverlayOptions` 物件。
3. 將物件指派給名為 `androidOverlay` 的變數。

```
val androidOverlay = GroundOverlayOptions()
```

4. 使用 `BitmapDescriptorFactory.fromResource()` 方法，從下載的圖片資源建立 `BitmapDescriptor` 物件。
5. 將產生的 `BitmapDescriptor` 物件傳遞至 `GroundOverlayOptions` 物件的 `image()` 方法。

```
val androidOverlay = GroundOverlayOptions()  
    .image(BitmapDescriptorFactory.fromResource(R.drawable.android))
```

6. 建立 `float overlaySize`，以公尺為單位表示所需疊加層的寬度。在本例中，`100f` 的寬度效果良好。

呼叫 `position()` 方法，為 `GroundOverlayOptions` 物件設定 `position` 屬性，並傳入 `homeLatLng` 物件和 `overlaySize`。

```
val overlaySize = 100f  
val androidOverlay = GroundOverlayOptions()  
    .image(BitmapDescriptorFactory.fromResource(R.drawable.android))  
    .position(homeLatLng, overlaySize)
```

7. 對 `GoogleMap` 物件呼叫 `addGroundOverlay()`，並傳入 `GroundOverlayOptions` 物件。

```
map.addGroundOverlay(androidOverlay)
```

8. 執行應用程式。

9. 將 `zoomLevel` 的值變更為 18f，即可看到 Android 圖片疊加在畫面上。



步驟 8 · 約 0 分鐘

8. 工作：啟用位置追蹤功能

使用者經常會透過 Google 地圖查看目前所在位置。如要在地圖上顯示裝置位置，可以使用[位置資料圖層](#)。

位置資料圖層會在 Google 地圖上新增「我的位置」圖示。



使用者輕觸按鈕後，地圖會以裝置所在位置為中心。如果裝置未移動，地圖上會以藍點標示位置；如果裝置正在移動，則會顯示為藍色箭號圖示。

在這項工作中，您會啟用位置資料層。

步驟：要求位置存取權

如要在 Google 地圖中啟用位置資訊追蹤功能，只需要一行程式碼。不過，您必須確保使用者已授予位置資訊權限 (使用執行階段權限模型)。

在這個步驟中，您會要求位置存取權，並啟用位置追蹤功能。

1. 在 `AndroidManifest.xml` 檔案中，確認是否已存在 `FINE_LOCATION` 權限。選取 Google 地圖範本時，Android Studio 會插入這項權限。

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

2. 在 `MapsActivity` 中，建立 `REQUEST_LOCATION_PERMISSION` 類別變數。

```
private val REQUEST_LOCATION_PERMISSION = 1
```

3. 如要檢查是否已授予權限，請在 `MapsActivity` 中建立名為 `isPermissionGranted()` 的方法。在這個方法中，檢查使用者是否已授予權限。

```
private fun isPermissionGranted() : Boolean {  
    return ContextCompat.checkSelfPermission(  
        this,  
        Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED  
}
```

4. 如要在應用程式中啟用位置資訊追蹤功能，請在 `MapsActivity` 中建立名為 `enableMyLocation()` 的方法，這個方法不採用任何引數，且不會傳回任何內容。並檢查是否包含 `ACCESS_FINE_LOCATION` 權限。如果授予權限，請啟用位置資訊圖層。否則請提出權限要求。

```
private fun enableMyLocation() {
    if (isPermissionGranted()) {
        map.isMyLocationEnabled = true
    }
    else {
        ActivityCompat.requestPermissions(
            this,
            arrayOf<String>(Manifest.permission.ACCESS_FINE_LOCATION),
            REQUEST_LOCATION_PERMISSION
        )
    }
}
```

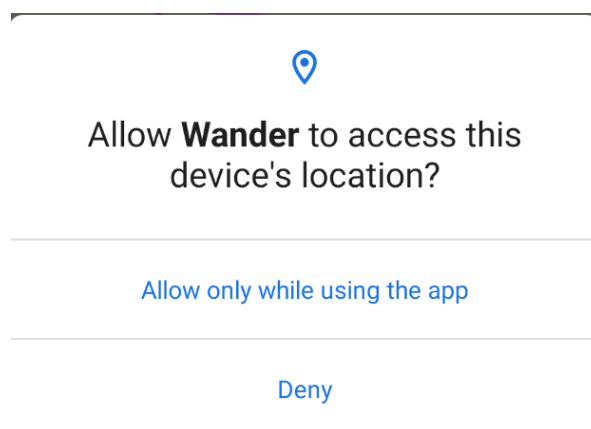
4. 從 `onMapReady()` 回呼呼叫 `enableMyLocation()`，即可啟用位置資訊圖層。

```
override fun onMapReady(googleMap: GoogleMap) {
    ...
    enableMyLocation()
}
```

5. 覆寫 `onRequestPermissionsResult()` 方法。檢查 `requestCode` 是否等於 `REQUEST_LOCATION_PERMISSION`。如果有的話，表示已授予權限。如果已授予權限，請檢查 `grantResults` 陣列的第一個位置是否包含 `PackageManager.PERMISSION_GRANTED`。如果真是如此，請撥打 `enableMyLocation()`。

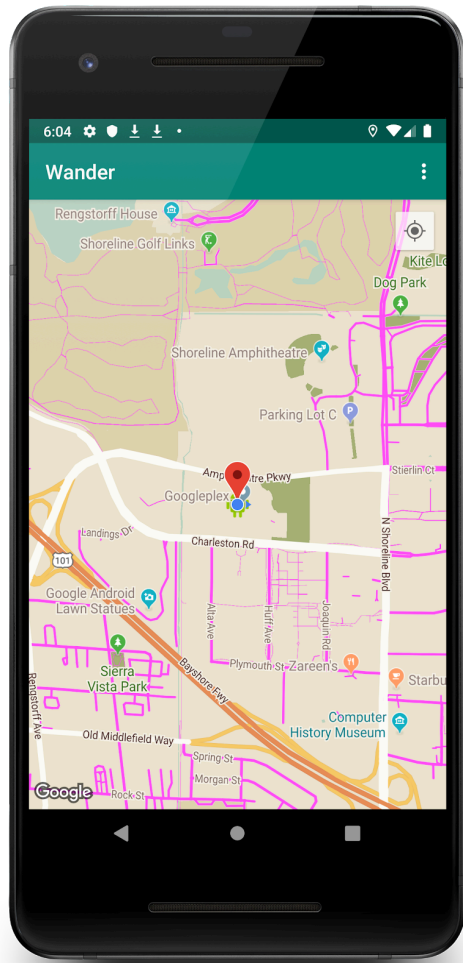
```
override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<String>,
    grantResults: IntArray) {
    if (requestCode == REQUEST_LOCATION_PERMISSION) {
        if (grantResults.contains(PackageManager.PERMISSION_GRANTED)) {
            enableMyLocation()
        }
    }
}
```

6. 執行應用程式。系統應會顯示對話方塊，要求存取裝置的位置資訊。請允許權限。



注意：如果您執行的 API 級別不是 29，這個對話方塊的外觀可能會有所不同。

地圖現在會以藍點顯示裝置的目前位置。請注意，當中會有位置按鈕。如果移動地圖，使中心點不再是你的位置，點選這個按鈕即可將地圖中心點移回裝置所在位置。



注意：在模擬器上執行應用程式時，可能無法使用位置資訊。如果尚未透過模擬器設定設定位置，就無法使用位置按鈕。

步驟 9 · 約 0 分鐘

9. 解決方案程式碼

下載完成的程式碼研究室程式碼。

```
$ git clone https://github.com/googlecode/andriod-kotlin-geo-maps
```

另外，您也可以下載存放區為 ZIP 檔案，然後解壓縮並在 Android Studio 中開啟。

注意：您必須在解決方案程式碼中加入 API 金鑰，否則地圖不會顯示。

步驟 10 · 約 0 分鐘

10. 摘要

恭喜！您已在 Android Kotlin 應用程式中加入 Google 地圖，並設定地圖樣式。

步驟 11 · 約 0 分鐘

11. 瞭解詳情

Android 開發人員說明文件：

- [開始使用](#)
- [加入含有標記的地圖](#)
- [地圖物件](#)
- [新增樣式化地圖](#)
- [街景服務](#)
- [區域疊加層](#)

參考說明文件：

- [GoogleMap](#)
 - [SupportMapFragment](#)
 - [SupportStreetViewPanoramaFragment](#)
-

步驟 12 · 約 0 分鐘

12. 下一個程式碼研究室

如要查看本課程其他程式碼研究室的連結，請參閱 [Android Kotlin 進階功能程式碼研究室登陸頁面](#)。
